

The Background Runtime

Why Executable Conversation Still Depends on What It Cannot Compile

Watson Hartsoe

Working Paper — AI-augmented research process

18 August 2026

Evidence, prompts, and making history preserved.

Abstract

Systems built from the language/action tradition offer a rare case in which social theory became executable machinery. The ActionWorkflow architecture and the Coordinator patent did not merely describe requests, promises, completions, and satisfactions: they represented conversational roles and incompletions as machine state and calculated permitted next moves. Yet the same sources acknowledge that conditions of satisfaction depend on shared assumptions and standard practices, while Suchman shows that the force of an utterance can be interactionally achieved rather than fixed in advance. This paper argues that executable social theory therefore has two semantically distinct layers: transition semantics, which a machine can enforce, and background semantics, which determines what those transitions mean and whether they are socially warranted. When software hides the second layer while enforcing the first, a descriptive ontology becomes a governance mechanism. The resulting problem is not that culture cannot be formalized, but that formalization must make its background dependencies, authorization conditions, and avenues for revision explicit.

Keywords: language/action perspective; workflow; speech acts; CSCW; formalization; institutional computation; hermeneutics; organizational ontology

1 Introduction: The part that executes is not the whole semantics

A social theory becomes technically consequential at a particular crossing. Before the crossing, a category such as *request*, *promise*, *satisfaction*, or *obligation* is an analytic proposal about social action. After the crossing, software stores the category as state, computes which operations are valid from it, and changes what users can do next. The language/action systems preserved in this field make that crossing unusually visible. ActionWorkflow organized work around a four-phase loop of proposal, agreement, performance, and satisfaction; its implementation architecture included a workflow language interpreter, transaction database, processor, and agents. The Coordinator went further at the interface: it used finite-state machinery and participant roles to calculate permitted next moves and to track incompletions as persistent data. [1, 2]

It is tempting to call this a successful compilation of conversation. That description is only half right. The same ActionWorkflow paper states that agreement depends on a shared background

of assumptions and standard practices. The transition *REQUEST* $\rightarrow$ *PROMISE* can be machine-legible while the practical meaning of what has been promised remains partly tacit. Later DEMO work retains a similar structure: communicative acts are modeled as state-changing organizational events, yet their effects depend on shared culture, norms, and values. [1, 6]

The central argument of this paper is that executable conversation contains a split semantics. **Transition semantics** determine which formal move follows which state. **Background semantics** determine what the move means, what counts as satisfying it, who has standing to perform it, and whether participants recognize the resulting obligation. The first layer is amenable to finite-state execution. The second is distributed across history, practice, authority, interpretation, and interaction. Treating the first as if it exhausted the second is the point at which formalization becomes governance rather than mere representation.

This distinction matters beyond workflow software. Searle’s constitutive-rule schema makes institutional status look formally compact, but the operative consequence of a status depends on deontic powers and conditions of successful declaration. Bourdieu’s habitus is expressly not a finite rule book: regular action can be generated without obedience to explicit rules. Suchman pushes the problem directly into system design by arguing that communicative intent and illocutionary force are not always available as clean inputs prior to interaction. These sources do not show that formalization is impossible. They show that a formalism must say which part of social reality it executes and which part it presupposes. [7, 8, 3]

2 From speech-act category to machine transition

The strongest evidence for executable social theory in this archive is not metaphorical. ActionWorkflow describes a software architecture that maintains workflow state and calculates next actions. The patent behind the Coordinator specifies stored finite-state-machine tables, role-dependent permitted moves, and records of incompletions. A user’s conversational position is therefore not only represented in a database; it conditions the action space exposed by the interface. [1, 2]

This creates an operational chain:

social category $\rightarrow$ data representation $\rightarrow$ state transition $\rightarrow$ interface affordance $\rightarrow$
organizational consequence

The chain is significant because each stage adds force. A scholar may propose that promises create commitments. A program must decide what representation constitutes a promise, when that representation is valid, and what follows from it. An interface that only presents state-valid moves closes the loop: the ontology returns to the world as a constraint on action.

The Coordinator’s incompleteness tokens sharpen the point. An absence - the response or fulfillment that has not yet occurred - can be represented as persistent machine state. This is a powerful computational move. It gives an organization memory: an obligation remains visible and actionable even when the original interaction has passed. Yet persistence also freezes an interpretation. A token can remain open after participants would describe the obligation as waived, misunderstood, impossible, or unjustly assigned unless those possibilities are themselves represented. Machine memory therefore amplifies whatever ontology was chosen at encoding time. [2]

The distributed implementation adds another formal commitment. Because messages may arrive out of order, the system reorders them before applying state transitions. This is sensible

engineering where transport has scrambled an already determined conversational sequence. It is more controversial where the interaction itself is concurrent or retrospectively interpreted. A state machine consumes an order; social interaction may supply only a partial order or a contested one. The point is not that sequencing is wrong. It is that every executable semantics must distinguish repaired transmission order from imposed social chronology.

3 The background that makes the state meaningful

The ActionWorkflow sources contain their own limit case. Agreement is not exhausted by the fields stored in the workflow record; it relies on a shared background of assumptions and standard practices. The same formal transition can therefore be semantically underdetermined. Two parties may occupy the machine state AGREED while differing about urgency, adequate quality, acceptable substitutions, tacit authority, or what completion will look like. [1]

Gadamer’s hermeneutic account of conversation offers a useful contrast, not because ActionWorkflow is secretly Gadamerian but because it clarifies what the protocol leaves outside. Genuine conversation can develop in ways not led by either participant alone; understanding changes through the encounter. Suchman brings a similar issue into CSCW at the level of utterance force. If the significance of a communicative move can be interactionally produced, then a system that asks the user to classify the move in advance may be demanding as input something that the interaction has not yet settled. [3]

The archived zettels call the omitted layer a *background runtime*. The phrase is compiler-created, not source terminology, but it names a concrete dependency: explicit transitions call upon tacit norms, histories, and shared practices in order to acquire social meaning. The runtime analogy becomes useful only if it remains asymmetric. Background is not another hidden deterministic rule table waiting to be extracted. It may include revisable judgments, embodied dispositions, institutional roles, contested conventions, and local histories that are not reducible to one representational type.

Bourdieu’s account of habitus is important here precisely because it blocks an easy computational solution. He describes durable dispositions capable of generating regular practices without obedience to explicit rules. A system that responds to the discovery of tacit background by converting every background condition into another explicit rule may therefore destroy the mechanism it hoped to capture. The challenge is not merely to add more rules. It is to represent the difference between rule, disposition, interpretation, recognition, and institutional authority. [8]

4 When a descriptive ontology becomes governance

Suchman’s political critique becomes sharper once read beside the patent. Her objection is not only that speech-act categories are descriptively incomplete. It is that categories embedded in organizational software can become ordering devices. The patent makes the mechanism concrete: conversation type, current state, role, prior moves, and incompletions determine the list of permitted moves shown to a participant. A category that once described an action now helps decide whether that action is available. [3, 2]

This is the transition from representation to governance. The important variable is not formalization alone but **deployment semantics**: Is the category optional or mandatory? Who may revise it? Can a participant refuse classification? What consequence follows from a token remaining

open? Are alternative descriptions visible? Does the system merely help a user remember a promise, or does managerial evaluation depend on the same machine state?

Winograd’s response to Suchman is valuable because it prevents the argument from collapsing into a prohibition on formal models. A model can be knowingly partial and still be useful for coordination. The political question appears when the limits of the representation are forgotten at deployment time. A workflow ontology can function as optional scaffolding in one setting and as compulsory accountability infrastructure in another. [4, 5]

The distinction suggests a stronger engineering principle than “do not formalize culture.” Every formal category that can constrain action should carry an account of its authority, revisability, and escape conditions. In programming-language terms, a transition function is not enough; the system also needs semantics for contesting the transition itself.

5 Institutional status needs more than a counts-as operator

Searle’s “X counts as Y in C” schema offers one of the cleanest formal shapes in social ontology. It is also a useful demonstration of what compact notation can conceal. The schema represents a status assignment, but executable institutional systems must additionally decide who is authorized to make the assignment, under what conditions it succeeds, what deontic powers follow, and how invalid or disputed declarations are represented. [7]

The problem is visible in ordinary examples. The same words can fail to marry, appoint, fire, promise, or authorize when spoken by an actor without standing. If authority is placed entirely inside C, the notation remains compact by hiding the most politically consequential variable in its context parameter. If authority is represented as another recursively assigned status, the system must explain how that chain acquires stability. If authority ultimately depends on coercion, recognition, organization, or material control, the institutional semantics need connections to mechanisms outside the counts-as rule.

This is not a refutation of constitutive rules. It is a design warning. An executable institutional language should make the context parameter decomposable and inspectable. At minimum, status-changing operations require explicit authorization predicates, resulting permissions and obligations, provenance, failure states, and procedures for challenge. The archived graph’s recurring ghost **authority** is therefore not a missing ornament. It is a missing runtime component.

6 A four-layer architecture for executable social theory

The evidence supports a layered architecture rather than a single universal grammar of culture. The layers are analytically separable even when an implementation interleaves them.

Transition layer. Represents explicit state, roles, commitments, event order, and permitted operations. This is where the Coordinator and ActionWorkflow are strongest.

Interpretive layer. Represents the background on which conditions of satisfaction, illocutionary force, relevance, and equivalence depend. Its values may be uncertain, distributed, or retrospectively revised.

Authorization/deontic layer. Represents who may confer a status and what rights, duties, permissions, prohibitions, or liabilities follow. Searle’s institutional ontology is most useful here, but computational realization requires more machinery than the counts-as schema alone.

Provenance/revision layer. Represents who supplied a classification, what evidence supported it, what alternatives were excluded, and how the classification can be contested or changed. Suchman’s critique makes this layer politically necessary; the archive’s own lossless-zettel discipline makes the same point epistemically.

The architecture is deliberately plural. Bourdieu warns against reducing practical regularity to rule obedience. Sperber’s cultural epidemiology likewise treats transformation rather than perfect replication as normal. Luhmann shifts the elementary operation of social systems to communication and treats state as recursively history-dependent. Their incompatibilities should not be erased. Their common lesson is narrower: social regularity cannot safely be equated with execution of a fixed explicit rule set.

7 What this changes about cultural programming

A cultural programming language should not be judged only by whether it has syntax and executable transitions. The harder question is whether it exposes the semantics it has *not* compiled. If a language can say **PROMISE**, it should also be able to represent that the meaning, authority, or conditions of satisfaction are contested. If it can set an obligation to **OPEN**, it should distinguish **OPEN** from **DISPUTED**, **WAIVED**, **MISUNDERSTOOD**, **IMPOSSIBLE**, and externally imposed. If it can localize the label of a move across communities, it should not assume that the underlying transition semantics are culturally invariant merely because the vocabulary can be translated.

This shifts the design target from a complete cultural machine to an executable research object with visible boundaries. The language can formalize a transition while keeping its background as an explicit dependency. It can run multiple interpretations against the same event history. It can preserve rival ontologies instead of collapsing them. It can expose when the formal possibility space excludes a socially available move. And it can record the remainder produced by formalization rather than treating that remainder as noise.

The result is still computation. But it is computation whose operational clarity does not pretend to settle the social theory that made the operation possible.

8 Limitations and unresolved territory

This paper is grounded in a finite recovered field. Eighty immutable cards are available as exact payloads from a previously generated archive, but that archive itself states that they are canonicalized exports rather than guaranteed verbatim reconstructions of every earlier conversation zettel. Five earlier research families - Axelrod, Generative Agents, Weil/Lévi-Strauss, Leach, and Parsons - survive only in a recovery manifest and are not treated here as full card evidence.

The paper also does not establish that the Coordinator’s actual organizational deployments were uniformly disciplinary, nor that participants experienced its categories in the way a formal analysis predicts. That would require empirical use studies and archival evidence beyond the current field. “Background runtime” is a compiler-created connective concept, not terminology claimed from Gadamer, Suchman, or the ActionWorkflow designers. Finally, the four-layer architecture is an engineering wager derived from collisions among the sources; it has not yet been validated as a complete theory of cultural computation.

The next discriminating work is empirical and technical. Recover the full Coordinator transition tables and surviving source code; compare them with naturally occurring conversations; collect cases in which participants dispute conditions of satisfaction or the classification of an utterance; and implement a runtime in which interpretations can be revised without rewriting the original event history.

9 Conclusion

The language/action systems are valuable because they demonstrate that social concepts can become real machine semantics. Requests, promises, incompletions, and satisfaction were not merely philosophical language; they became stored state, transition constraints, and user-visible options. But the same historical case shows why successful execution is not the same as semantic completeness. Formal transitions depend on backgrounds that may remain tacit, contested, embodied, or interactionally produced.

The political consequence follows directly. When a machine enforces a transition while hiding the background that makes the transition meaningful, its ontology becomes an institution. The right response is neither to abandon formalization nor to pretend that all tacit context can be compiled into more rules. It is to design formal systems that expose their background dependencies, authorization conditions, revision paths, and representational losses. The executable part of culture should be allowed to execute. It should not be allowed to impersonate the whole.

Appendix A — Assembly Instrument

Exact package path: `_PROMPTS/001__SLIPCASE_15.55-AM.txt`.

The same instrument is embedded in `index.html`. Its byte-for-byte equivalence to the originating chat turn is not independently machine-verifiable because the build environment exposes files, not raw message bytes; this limitation is recorded in `_SLIPCASE/VERIFICATION.txt`.

Appendix B — Making History

See `000__MAKING_HISTORY.txt` and `background-runtime__MAKING_HISTORY.txt`. Immutable zettel payloads were preserved from the prior archive without rewriting. Graph structure, MOCs, arrangements, paper prose, source map, reader, and visual design are derived by GPT-5.6 Sol in this run.

Appendix C — Replication Path

Open `index.html`, inspect the manifest and immutable card payloads, then read `000__RETURN_PATH.txt` and `000__REBUILD.txt`. `_SLIPCASE/rebuild.py` verifies card hashes and rebuilds core machine-readable indexes from the root card box.

References

- [1] Raul Medina-Mora, Terry Winograd, Rodrigo Flores, and Fernando Flores. “The Action Workflow Approach to Workflow Management Technology.” In *Proceedings of CSCW 1992*, 281–288. ACM, 1992. DOI: <https://doi.org/10.1145/143457.143530>.
- [2] Carlos F. Flores et al. *Method and Apparatus for Structuring and Managing Human Communications by Explicitly Defining the Types of Communications Permitted Between Participants*. US Patent 5,208,748, 1993.
- [3] Lucy Suchman. “Do Categories Have Politics? The Language/Action Perspective Reconsidered.” *Computer Supported Cooperative Work 2* (1993): 177–190. DOI: <https://doi.org/10.1007/BF00749015>.
- [4] Terry Winograd. “Categories, Disciplines, and Social Coordination.” *Computer Supported Cooperative Work 2* (1994): 191–197. DOI: <https://doi.org/10.1007/BF00749016>.
- [5] Lucy Suchman. “Speech Acts and Voices: Response to Winograd et al.” *Computer Supported Cooperative Work 3* (1995): 85–95. DOI: <https://doi.org/10.1007/BF01305852>.
- [6] Jan L. G. Dietz. “Understanding and Modelling Business Processes with DEMO.” In *Conceptual Modeling – ER ’99*, 188–202. Springer, 1999. DOI: https://doi.org/10.1007/3-540-47866-3_13.
- [7] John R. Searle. “Constitutive Rules.” *Argumenta* 4(1) (2018): 51–54. DOI: <https://doi.org/10.14275/2465-2334/20187.sea>.
- [8] Pierre Bourdieu. *Outline of a Theory of Practice*. Trans. Richard Nice. Cambridge University Press, 1977.